Task Scheduling for Dependent Non-Preemptive Tasks: A Comprehensive Overview

Olivier

2025

Abstract

This article presents a comprehensive comparison of scheduling algorithms for dependent tasks, with a focus on minimizing the total execution time (makespan). We evaluated several approaches, including **Height**, **Depth**, **Blevel**, **Tlevel**, **Successors**, **Duration**, and the **Genetic Algorithm (GA)**. Our experiments, conducted on 180 graphs with 50 to 5000 tasks each, revealed that **Blevel-based schedules** consistently achieve the best performance, offering a significant improvement over other methods. The proposed enhancement using **Blevel with fuzzy interference** did not provide substantial benefits, making its implementation less justified.

The **Genetic Algorithm** demonstrated potential for achieving lower makespans but suffered from high computational costs and variability in performance. While parallelization and hybridization with techniques like **Local Search** or **Particle Swarm Optimization (PSO)** could improve its efficiency, further investigation is needed to make the GA competitive with Blevel-based schedules. This study highlights the strengths of Blevel as the current best approach and underscores the need for continued research into advanced optimization techniques for task scheduling.

Introduction

Task scheduling is a fundamental problem in computer science, particularly in the context of parallel computing and operating systems. The goal is to allocate tasks to processing/worker/machine units in a way that optimizes certain criteria, such as minimizing the total execution time (makespan). The problem of scheduling dependent tasks on parallel machines is NP-complete, meaning no known algorithm can solve it exactly in polynomial time for all instances. This implies that for large-scale practical cases, we rely on approximate solutions (heuristics or meta-heuristics) that provide good-quality results in reasonable time, rather than seeking an optimal solution that could require prohibitive computation time. In this article, we focus on **dependent non-preemptive tasks**, where:

- **Dependent tasks**: Tasks have dependencies, meaning some tasks cannot start until others have completed.
- **Non-preemptive tasks**: Once a task starts executing, it cannot be interrupted until it finishes.

The problem becomes more complex when tasks have varying durations and dependencies, requiring sophisticated algorithms to ensure efficient scheduling.

0.1 DAG Representation of the Problem

To model task dependencies, we use a **Directed Acyclic Graph (DAG)**. In this representation:

- Each node represents a task.
- A directed edge from task T_i to task T_j indicates that T_i must be completed before T_j can start.

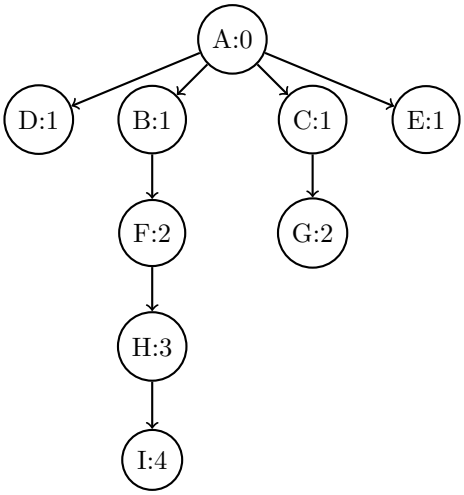
The acyclic nature of the graph is crucial because cycles would create circular dependencies, making it impossible to schedule the tasks. For example, if T_1 depends on T_2, and T_2 depends on T_1, neither task can start, leading to a deadlock.

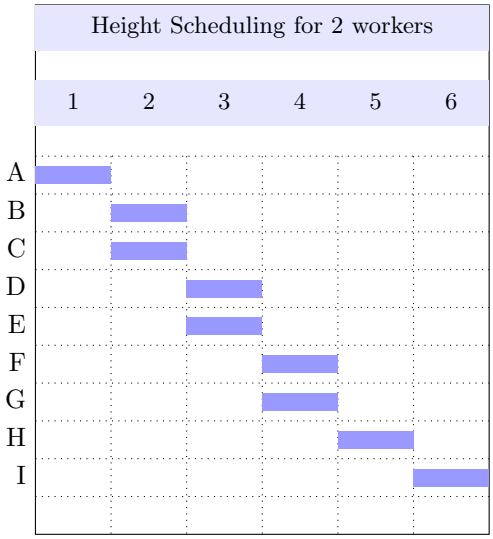
1 Scheduling Tasks with Unit Duration

We begin by considering tasks with **unit duration**, where each task takes the same amount of time to execute. This simplification allows us to focus on the dependency structure without considering varying task lengths.[**1**].

1.1 Label Height: A Top-Down Approach

The *label height* approach assigns a level to each task based on its position in the DAG. Tasks with no dependencies are assigned level 0, while tasks depending on others are assigned levels incrementally. This ensures that tasks are scheduled in a way that respects dependencies.





1.1.1 Pseudo-Code for Label Height

The following pseudo-code demonstrates the *label height* algorithm:

Algorithm 1 Label Height Algorithm

```
Initialize all tags to -1 (no tag)
k ← 0                                   ▷ Current iteration
n ← 0                                   ▷ Number of tagged tasks
while n < total number of tasks do
    Find all tasks ready to be tagged (no untagged predecessors)
    for each ready task do
        Assign tag k to the task
        n ← n + 1
    end for
    k ← k + 1
end while
```

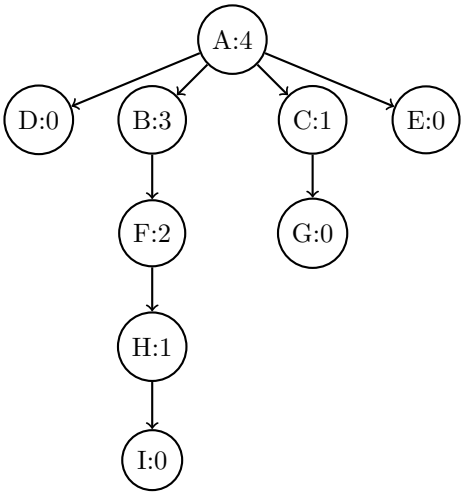
By processing tasks level by level, we ensure that all dependencies are respected. For example, tasks at level 0 are scheduled first, followed by tasks at level 1, and so on.

1.1.2 Limitations of Label Height

While the *label height* approach works well for unit-duration tasks, it is not optimal for tasks with varying durations. The algorithm does not account for the downstream impact of scheduling decisions, potentially leading to inefficient schedules.

1.2 Label Depth: A Bottom-Up Approach

To address the limitations of *label height*, we introduce *label depth*, which considers the downstream structure of the DAG. Instead of assigning levels based on predecessors, *label depth* assigns levels based on successors, providing a more comprehensive view of the task dependencies.



1.2.1 Pseudo-Code for Label Depth

The following pseudo-code demonstrates the *label depth* algorithm:

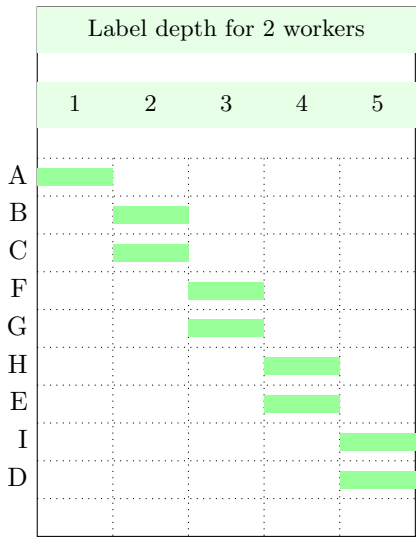
Algorithm 2 Label Depth Algorithm

```
Initialize all tags to -1 (no tag)
k ← 0                                   ▷ Current iteration
n ← 0                                   ▷ Number of tagged tasks
while n < total number of tasks do
    Find all tasks ready to be tagged (no untagged successors)
    for each ready task do
        Assign tag k to the task
        n ← n + 1
    end for
    k ← k + 1
end while
```

This approach ensures that tasks with the most downstream dependencies are scheduled first, reducing the overall makespan.

1.3 Hu's Algorithm

Hu's algorithmbuilds on the *label depth* approach but incorporates additional a ready list to provide multiple levels in one row.



1.3.1 Pseudo-Code for Hu's Algorithm

The following pseudo-code demonstrates Hu's algorithm:

Algorithm 3 Hu's Algorithm

```
Compute label depth for all tasks
Sort tasks in decreasing order of label depth
while there are unscheduled tasks do
    Select up to p ready tasks (tasks with all predecessors completed)
    Schedule the selected tasks on the processors
    Mark the scheduled tasks as completed
end while
```

Hu's algorithm is optimal for in-tree task graphs with p processors. This means that when the task dependencies form an in-tree structure (a directed acyclic graph where each task has at most one successor), Hu's algorithm guarantees a schedule with the minimum possible makespan when using p processors.

2 Scheduling Tasks with Arbitrary Durations

In real-world scenarios, tasks often have arbitrary durations, making scheduling more complex. To handle this, we use a **ready queue** implemented as a linked list, which dynamically manages tasks based on their dependencies and priorities. Tasks are inserted into the queue according to a chosen sorting criterion (e.g., depth or height), ensuring efficient scheduling.

2.1 Linked List: A Dynamic Data Structure

A **linked list** is a linear data structure where each element, called a **node**, contains a task and a pointer to the next node. Unlike arrays, linked lists allow for efficient dynamic memory allocation, making them ideal for managing tasks in a ready queue where tasks are frequently inserted or removed. This flexibility is crucial for scheduling algorithms, as it enables real-time updates to the task queue without requiring costly reallocations.

Below is a minimal pseudo-code for key linked list operations:

Algorithm 4 Linked List Operations

```
function CREATE_NODE(Task* task)
    Allocate memory for a new node
    Assign the task and set next pointer to NULL
    return the node
end function
function INSERT_SORTED(TaskNode** head, Task* task, int* tags)
    Create a new node using create_node
    Traverse the list and insert the node in descending order of tags
end function
function REMOVE_NODE(TaskNode** head, TaskNode* node)
    Update pointers to bypass the node
    Free the memory of the removed node
end function
```

2.2 Scheduling Algorithm with Arbitrary Durations

Using the linked list, we implement a scheduling algorithm that dynamically manages the ready queue and assigns tasks to processors based on their dependencies and priorities. Below is the pseudo-code for the scheduling algorithm:

Algorithm 5 Scheduling Algorithm with Arbitrary Durations

```
function SCHEDULE_TASKS(Graph* graph, int p, int** start_dates, int** finish_dates)
    Compute task priorities (e.g., using label_depth)
    Initialize task states, start dates, and finish dates
    Create a task queue and sort it by the chosen criterion
    Initialize a ready queue with tasks that have no dependencies
    Initialize processors to idle
    Set the current time to 0
    while there are uncompleted tasks do
        Assign tasks from the ready queue to idle processors
        Increment the current time
        Check for finished tasks and update the ready queue
    end while
    return the maximum finish time
end function
```

2.3 Explanation of the Algorithm

The algorithm works as follows:

- Tasks are sorted by a chosen criterion (e.g., depth or height) and inserted into a linked list.
- The ready queue is dynamically updated as tasks become ready (i.e., all dependencies are satisfied).
- Tasks are assigned to idle processors, and their start and finish times are recorded.
- The algorithm continues until all tasks are completed, ensuring dependencies are respected and the makespan is minimized.

This approach is flexible and can be adapted to different sorting criteria, making it suitable for a wide range of scheduling problems with arbitrary task durations.

3 Prioritization Strategies for Task Scheduling

To efficiently schedule tasks with dependencies, various prioritization strategies can be employed. These strategies determine the order in which tasks are assigned to processors. Below, we discuss five common approaches: **height**, **depth**, **highest number of successors**, **highest duration**, **blevel** and **tlevel**. We also introduce a hybrid approach called **blevel_height**, which combines **blevel** with a fuzzy inference engine for tie-breaking.

3.1 Height

The **height** of a task is the length of the longest path from a source task (a task with no predecessors) to the task. Tasks with greater height are prioritized to minimize delays in the overall schedule.

3.2 Depth

The **depth** of a task is defined as the length of the longest path from the task to a sink task (a task with no successors). Tasks with higher heights are prioritized to ensure that critical paths are completed early.

3.3 Highest Number of Successors

Tasks with the **highest number of successors** are prioritized to reduce the waiting time for dependent tasks. This approach aims to maximize parallelism.

3.4 Highest Duration

Tasks with the **longest duration** are prioritized to ensure that they start early, preventing them from becoming bottlenecks in the schedule.

3.5 Blevel

The **blevel** (bottom level) of a task is the length of the longest path from the task to a sink task, including the task's own duration. It provides a more accurate measure of a task's criticality compared to depth. Below is the pseudo-code for computing the **blevel**:

Algorithm 6 Compute Blevel

```
1:  function LABEL_BLEVEL(Graph* graph, int** tags_blevel_ptr)
2:      Initialize all tags to -1
3:      n ← 0                                          ▷ Number of tagged tasks
4:      while n < total number of tasks do
5:          Find tasks ready to be tagged (no untagged successors)
6:          for each ready task do
7:              Compute the maximum tag of its successors
8:              Set the task's tag to max_tag + duration
9:              n ← n + 1
10:         end for
11:     end while
12: end function
```

3.6 Tlevel

The **tlevel** (top level) of a task is the length of the longest path from a source task to the task, including the task's own duration. It represents the earliest possible start time for the task.

3.7 Blevel_Height: A Hybrid Approach

When tasks have the same **blevel**, a tie-breaking mechanism is needed. The **blevel_height** approach uses a **triangular fuzzy inference engine** to prioritize tasks based on their **height** and **duration**. This method combines the advantages of **blevel** and **height** while considering task durations. [**2**].

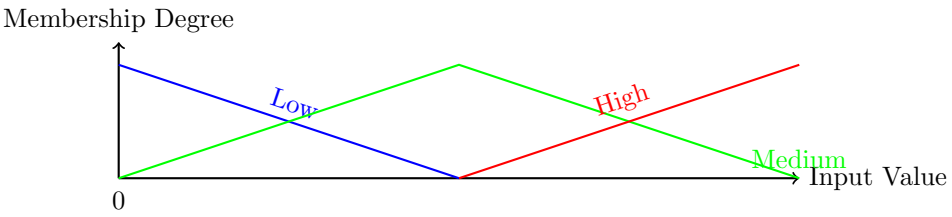
3.7.1 Triangular Fuzzy Inference Engine

A **Triangular Fuzzy Inference Engine** is a decision-making tool that uses fuzzy logic to handle imprecise or uncertain information. Unlike traditional binary logic (where values are either 0 or 1), fuzzy logic allows for partial membership in multiple categories, represented by **membership functions**. In this context, a **triangular membership function** is used to map input values (e.g., task height or duration) to a degree of membership in predefined categories (e.g., low, medium, high).

Key Concepts

- **Membership Functions**:

- A triangular membership function is defined by three parameters: the start, peak, and end of the triangle.
- For example, the membership function for "low height" assigns a value of 1.0 to tasks with the smallest height and gradually decreases to 0.0 as the height increases.



Triangular Membership Functions

- **Fuzzy Rules**:
  - Rules are defined to combine membership values from multiple inputs (e.g., height and duration) into a single output (priority score).
  - Example rule: If height is **low** and duration is **medium**, then priority is **high**.
- **Defuzzification**:
  - The output of the fuzzy rules is a set of weighted values. Defuzzification combines these values into a single crisp priority score.

Why Use a Triangular Fuzzy Inference Engine?

- It provides a flexible way to handle ambiguous or overlapping criteria (e.g., when tasks have similar heights or durations).
- It allows for nuanced decision-making, which is particularly useful for tie-breaking in task scheduling.

Example Membership Functions Below are the triangular membership functions for task height and duration:

Algorithm 7 Membership Functions

```
function MEMBERSHIP_LOW_HEIGHT(float height, float max_height)
    if height ≤ max_height/2 then
        return 1.0 − (height/(max_height/2))
    else
        return 0.0
    end if
end function
function MEMBERSHIP_MEDIUM_HEIGHT(float height, float max_height)
    if height ≤ max_height/2 then
        return height/(max_height/2)
    else
        return 2.0 − (height/(max_height/2))
    end if
end function
function MEMBERSHIP_HIGH_HEIGHT(float height, float max_height)
    if height ≥ max_height/2 then
        return (height − (max_height/2))/(max_height/2)
    else
        return 0.0
    end if
end function
```

How It Works

1. **Input Evaluation**:
   - For each task, evaluate its height and duration using the membership functions.

2. **Rule Application**:
    - Apply fuzzy rules to combine the membership values into a priority score.
3. **Defuzzification**:
    - Aggregate the results of all rules to compute a final priority score for the task.

This approach ensures that tasks with similar **blevel** values are prioritized based on their height and duration in a systematic and fair manner. —

3.8 Comparison of Scheduling Algorithms

To evaluate the performance of the different scheduling algorithms, we conducted experiments using 180 graphs, each containing 1000 tasks [**3**].. The number of workers is randomly between 2 and 5 % of tasks. The results are compared based on two metrics: (1) the **ratio of improvement** relative to the **height-based schedule** (chosen as the baseline because it was the first algorithm discovered), and (2) the **execution speed** of each algorithm.

3.9 Ratio of Improvement

The ratio of improvement is calculated as:

$$\text{Ratio} = 100 * (1 - \frac{\text{Schedule of Algorithm X}}{\text{Schedule of Height}})$$

A positive ratio indicates an improvement over the height-based schedule. The results are as follows:

- **Blevel Ratio**: Mean = 2.432460, Standard Deviation = 4.006291
- **Depth Ratio**: Mean = 1.792040, Standard Deviation = 3.047140
- **Blevel-Height Ratio**: Mean = 2.430791, Standard Deviation = 4.004351
- **Tlevel Ratio**: Mean = -1.075348, Standard Deviation = 2.179151
- **Successors Ratio**: Mean = -0.142785, Standard Deviation = 1.747450
- **Duration Ratio**: Mean = -0.286566, Standard Deviation = 1.346814

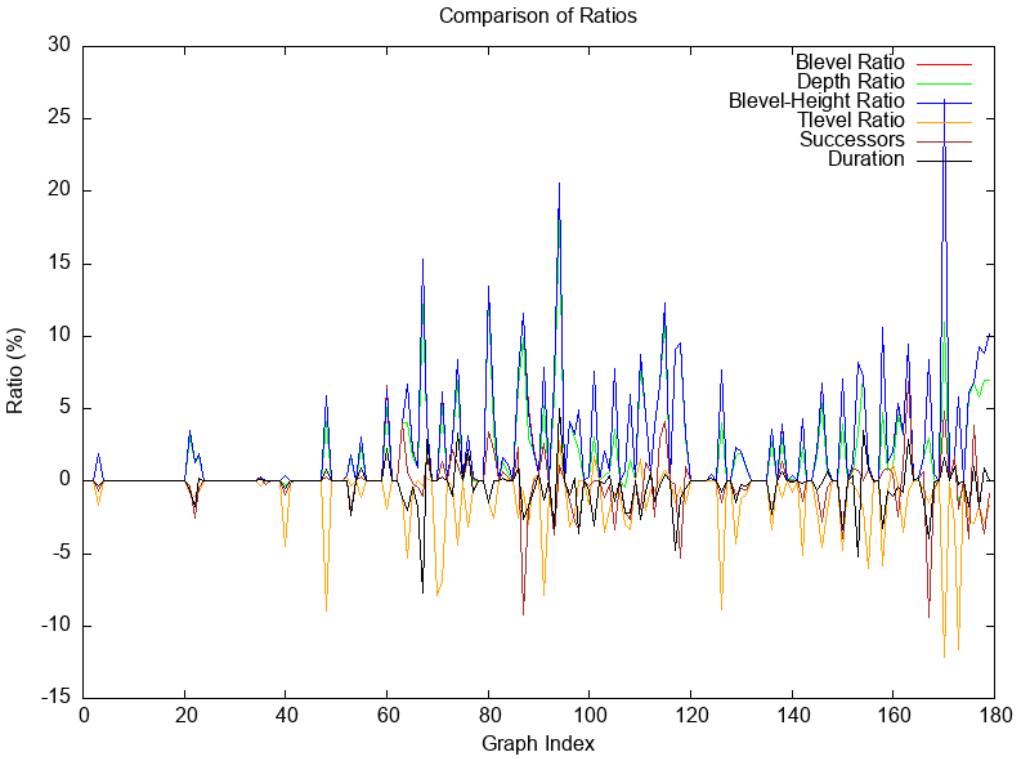


Figure 1: Ratio of improvement for each algorithm relative to the height-based schedule.

From the results, we observe that **Blevel** performs the best, with a significant improvement over the height-based schedule. The improvement from **Blevel-Height** is not significantly different from Blevel, indicating that the additional complexity of the fuzzy inference engine does not provide a substantial benefit. On the other hand, **Tlevel**, **Successors**, and **Duration** show negative ratios, indicating that they perform worse than the height-based schedule.

3.10 Execution Speed

The execution speed of all algorithms is similar, with an average difference of less than 0.1 seconds. This small difference is acceptable for practical applications, as it does not significantly impact the overall performance.

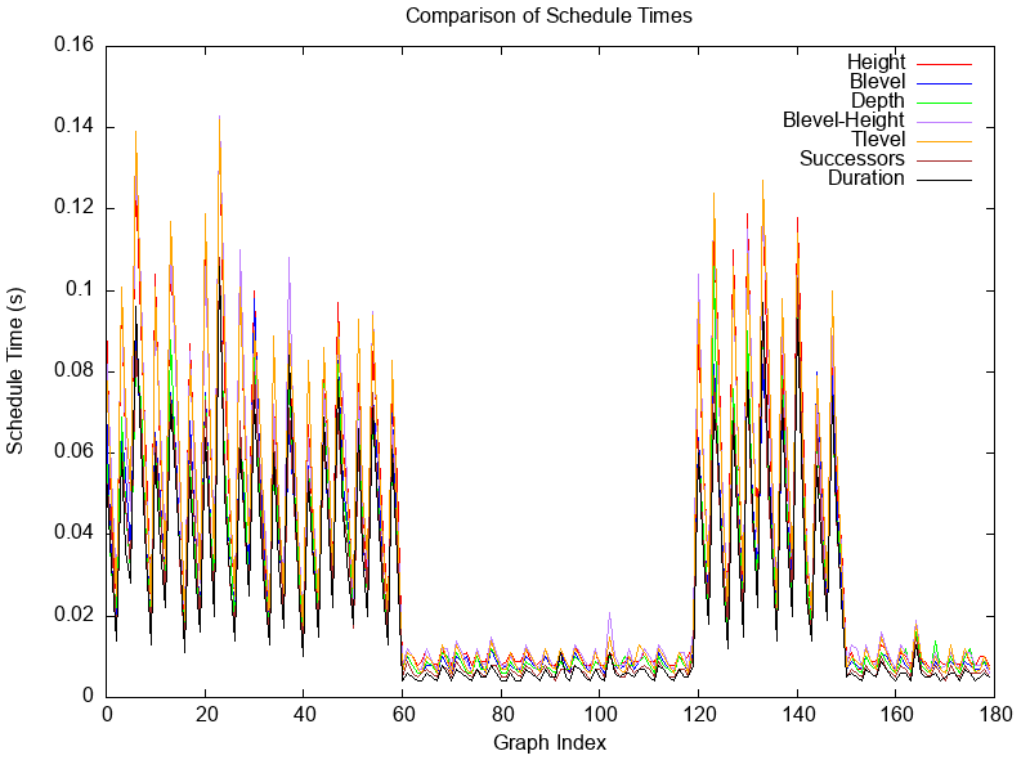


Figure 2: Execution speed of each algorithm for 1000.

Be careful: for 5,000 tasks, the sorting process takes more than 20 seconds, which is logical since the algorithm is based on the number of dependencies which is more than 160 000.

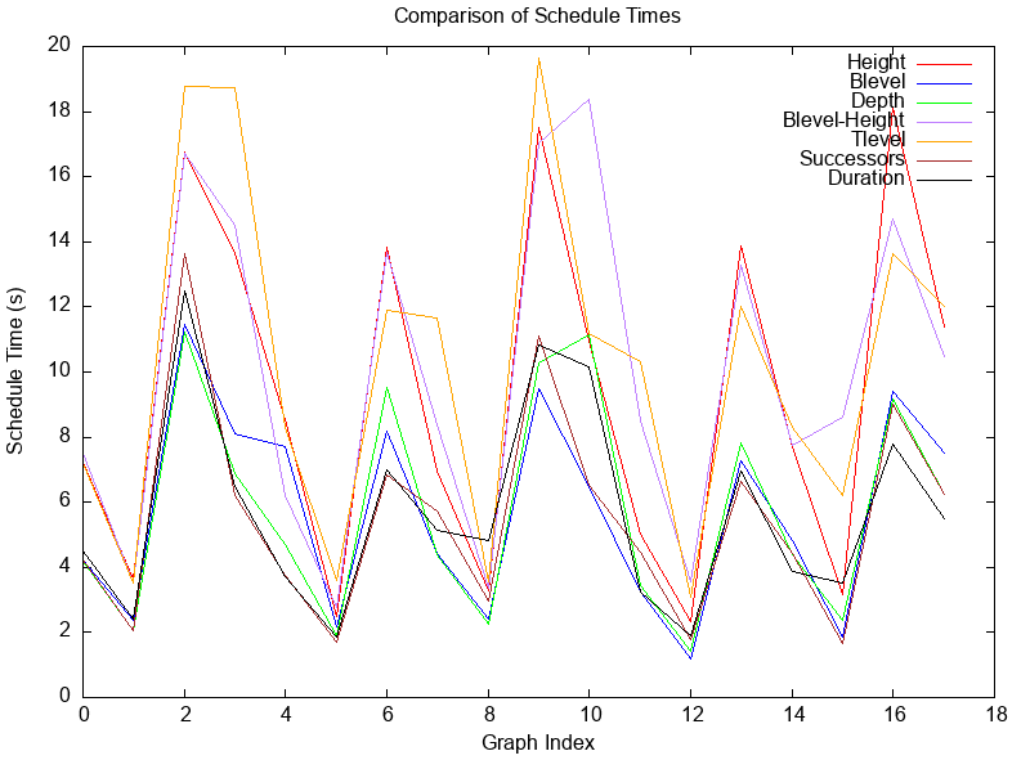


Figure 3: Execution speed of each algorithm for 5000.

3.11 Conclusion

The results demonstrate that **Blevel** is the most effective algorithm for scheduling tasks with dependencies, providing a significant improvement over the baseline height-based schedule. While **Blevel-Height** shows similar performance, its added complexity does not justify its use in this context. The execution speed of all algorithms is comparable, making the choice of algorithm dependent on the desired scheduling performance rather than computational efficiency.

4 Genetic Algorithm for Task Scheduling

The **Genetic Algorithm (GA)** [4]. is an optimization technique inspired by natural evolution. Below, we outline the key steps of the GA implementation for task scheduling.

4.1 Key Steps of the Genetic Algorithm

The GA operates through the following steps:

1. **Representation of the Problem**:
   - A **chromosome** represents a solution and consists of 6 weights (8 bits each) corresponding to the following heuristics:
     - **Height**
     - **Depth**
     - **Successor**
     - **Blevel**
     - **Tlevel**
     - **Duration**
   - Each chromosome is a 48-bit string, where each 8-bit segment represents a weight for one heuristic.
2. **Initialization of the Population**:
   - A population of chromosomes is created randomly.
   - Each chromosome is initialized with random values between 0 and 255 for each heuristic weight.

3. **Fitness Evaluation**:
   - The fitness of a chromosome is calculated by scheduling tasks using its weights and computing the resulting makespan.
   - Tasks are prioritized using the weighted sum of the normalized heuristics :
   $$p(n) = (P_0 \cdot w_0) + (P_1 \cdot w_1) + (P_2 \cdot w_2) + (P_3 \cdot w_3) + (P_4 \cdot w_4) + (P_5 \cdot w_5)$$
   - The schedule is generated, and the makespan is returned as the fitness value.
4. **Selection**:
   - The best chromosomes (those with the lowest makespan) are selected for reproduction.
   - Common methods include **tournament selection**.
5. **Crossover**:
   - Two parent chromosomes are combined to create two offspring.
   - A **single-point crossover** is used: a random point is chosen, and segments after this point are swapped between the parents.
6. **Mutation**:
   - Each bit in a chromosome is flipped with a small probability (mutation rate).
   - This introduces diversity into the population and avoids local extremum.
7. **Replacement**:
   - The least fit chromosomes in the population are replaced with the new offspring.
8. **Termination**:
   - The algorithm stops when a termination condition is met, such as:
     - A maximum number of generations.
     - A satisfactory fitness value (e.g., a sufficiently low makespan).
     - Convergence of the population (chromosomes become too similar).

4.1.1 Pseudo-Code for the Genetic Algorithm

Below is a concise pseudo-code for the genetic algorithm:

Algorithm 8 Genetic Algorithm for Task Scheduling

```
Initialize population with random chromosomes
for each generation do
    Evaluate fitness of each chromosome
    Select parents using tournament selection
    Perform crossover to create offspring
    Apply mutation to offspring
    Replace least fit chromosomes with offspring
end for
Return the best chromosome (solution)
```

4.1.2 Fitness Calculation

The fitness function calculates the makespan of a schedule generated by a chromosome's weights. Below is a simplified version of the fitness calculation:

Algorithm 9 Fitness Calculation

```
function CALCULATE_FITNESS(Graph* graph, int p, Chromosome* chromosome)
    Decode chromosome to get heuristic weights
    Calculate task priorities using weighted heuristics
    Schedule tasks using a priority queue
    Compute the makespan of the schedule
    return the makespan as the fitness value
end function
```

4.2 Comparison of Genetic Algorithm with Other Schedulers

To evaluate the performance of the **Genetic Algorithm (GA)**, we conducted experiments on 180 graphs, each containing 50 tasks.The number of workers is randomly between 2 and 5 % of tasks. The GA was configured with a population size of 30 and run for 20 generations. The results were compared with other scheduling algorithms, including **Height**, **Depth**, **Blevel**, **Blevel-Height**, **Tlevel**, **Successors**, and **Duration**.

4.2.1 Experimental Setup

- **Number of Graphs**: 180
- **Tasks per Graph**: 50
- **GA Parameters**:
  - Population size: 30
  - Number of generations: 20
  - Crossover rate: 0.5
  - Mutation rate: 0.2
- **Comparison Metrics**:
  - Makespan (total execution time)
  - Execution speed (time to compute the schedule)

4.2.2 Results

The results show that the **Genetic Algorithm** performs not really better than the other scheduling algorithms in terms of makespan. However, its execution speed is considerably slower due to the computational overhead of evolving a population over multiple generations.

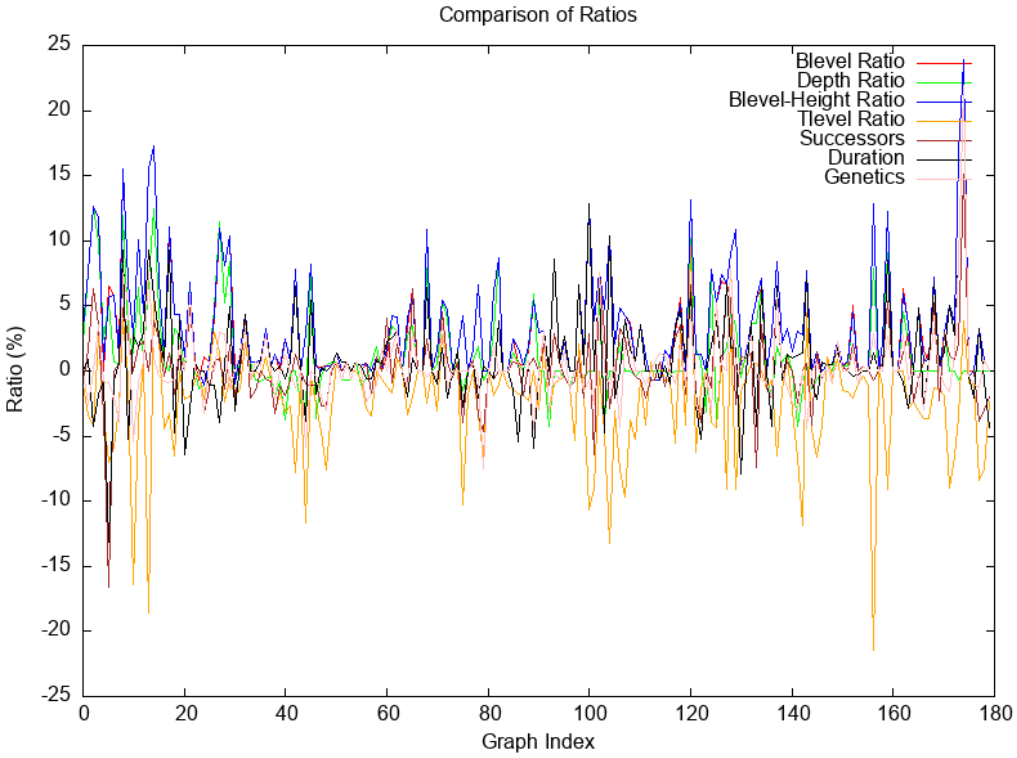


Figure 4: Comparison of makespan for different scheduling algorithms. The Genetic Algorithm (GA) achieves the lowest makespan.

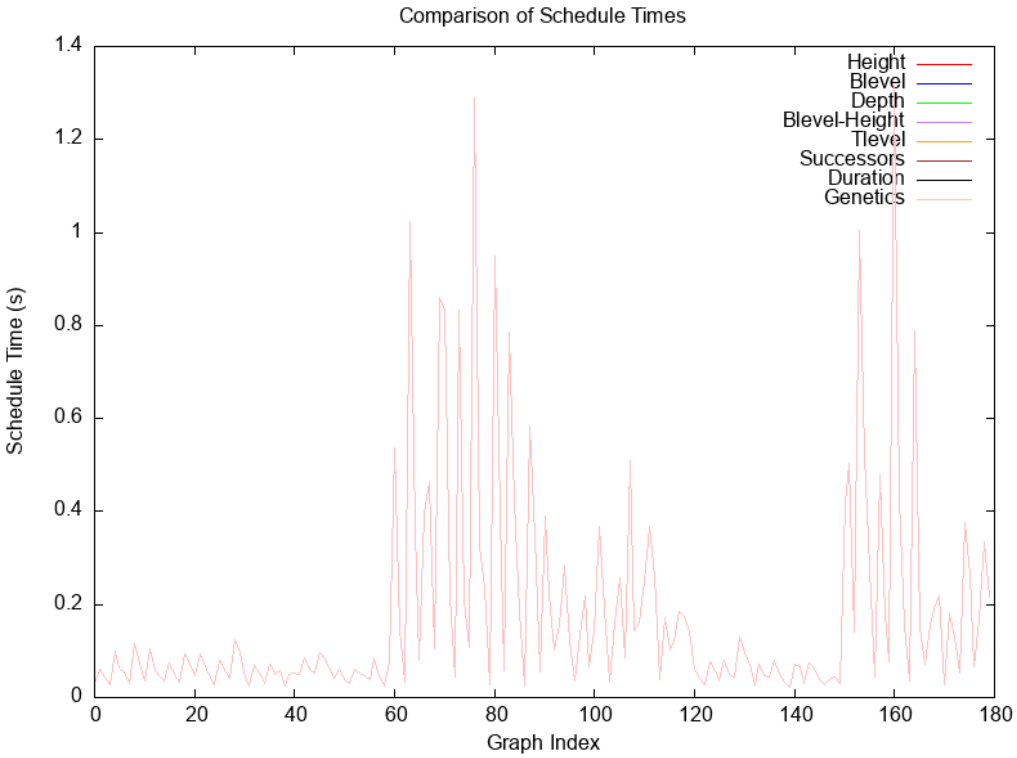


Figure 5: Comparison of speed for different scheduling algorithms. The blevel has the highest computational cost.

4.3 Performance Ratios

The performance of each algorithm is compared using the ratio of its makespan to the makespan of the **Height**-based schedule. A positive ratio indicates an improvement over the baseline, while a negative ratio indicates worse performance. The mean and standard deviation of the ratios are as follows:

- **Blevel Ratio**: Mean = 3.109067, Standard Deviation = 3.386987
- **Depth Ratio**: Mean = 1.109499, Standard Deviation = 2.772616
- **Blevel-Height Ratio**: Mean = 3.131460, Standard Deviation = 3.403894
- **Tlevel Ratio**: Mean = -2.336841, Standard Deviation = 4.003549
- **Successors Ratio**: Mean = 0.272274, Standard Deviation = 2.414842
- **Duration Ratio**: Mean = 0.610102, Standard Deviation = 3.339924
- **Genetics Ratio**: Mean = 0.246439, Standard Deviation = 2.283787

4.4 Key Observations

The GA failed which can be explained by a lack of generations or population. However the main point is that 50 tasks is too low , most of the schedule will gave the same amount and a selection can't be proceed. We conclude that GA needs to be used in large model but it's actually too slow to be practical.

4.5 Optimizing the Genetic Algorithm for Speed

To reduce the computational time of the **Genetic Algorithm (GA)**, we explored two main approaches: (1) reducing the population size and (2) reducing the number of generations. We conducted experiments with population sizes ranging from 10 to 30 and generations ranging from 2 to 10. Each configuration was tested on 30 random graphs with 50 tasks with random number of workers, and the results were averaged.

4.5.1 Experimental Setup

- **Population Sizes Tested**: 10-¿30
- **Generations Tested**: 2-¿10
- **Number of Graphs per Configuration**: 30
- **Tasks per Graph**: 50

4.5.2 Results

The results are visualized in the following plots:

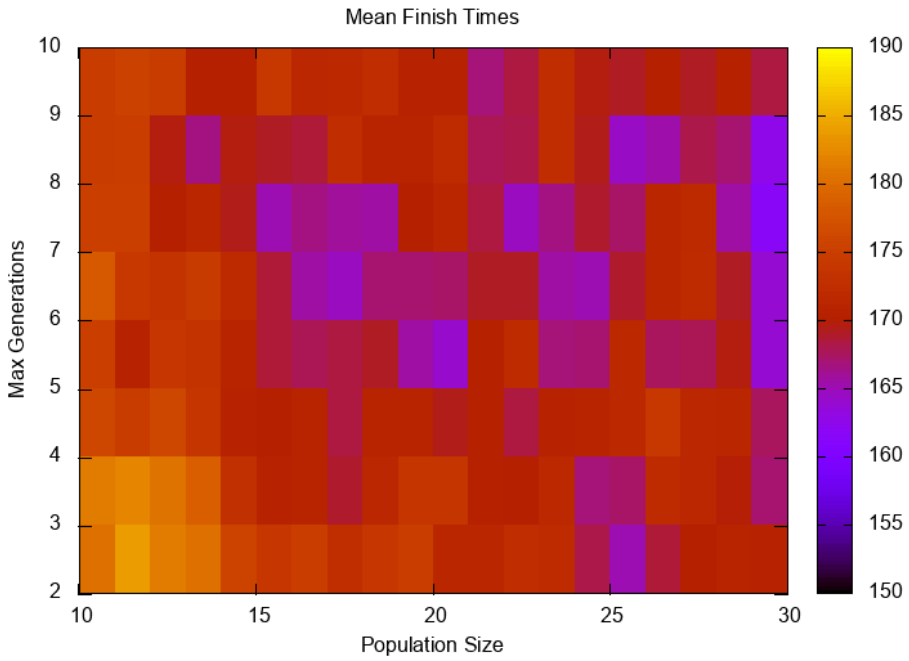


Figure 6: Makespan as a function of population size. A significant improvement in solution quality is observed at a population size of 20, which appears to be the turning point. Beyond this, the improvement diminishes.

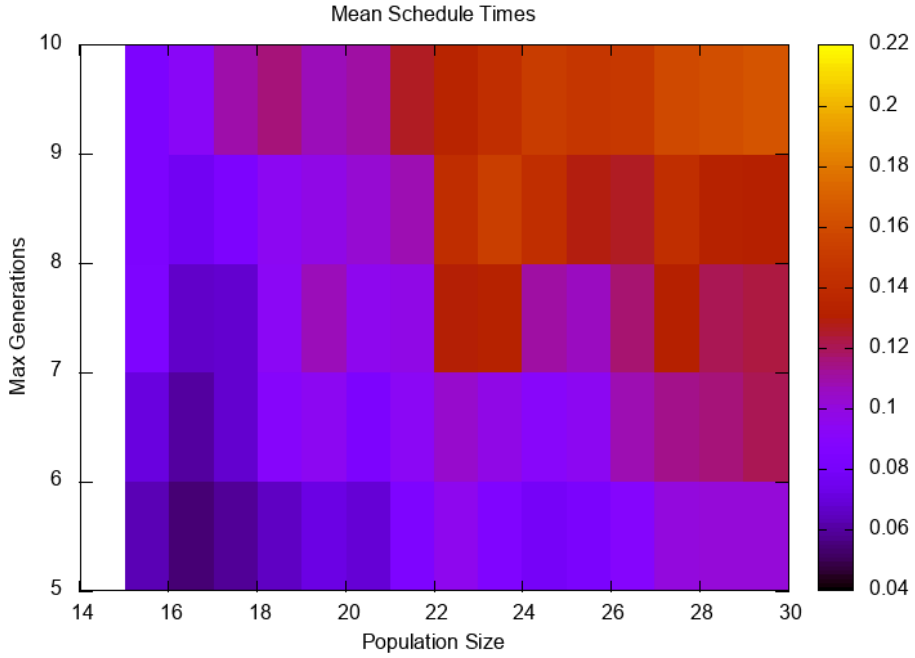


Figure 7: Execution speed as a function of the number of generations and population size. The relationship is linear fixing the other variable, indicating that reducing the number of generations does not significantly improve speed.

4.5.3 Key Observations

- **Population Size**:
  - A population size of 18 appears to be the turning point, where the solution quality begins to improve significantly.
  - Increasing the population size beyond 18 provides diminishing returns, as the solution quality is already near optimal.
- **Number of Generations**:
  - The relationship between the number of generations and execution speed is linear.
  - Reducing the number of generations does not significantly improve speed, as the computational cost per generation remains high.
- **Trade-off**:
  - While reducing the population size to 18 improves speed without significantly compromising solution quality, reducing the number of generations does not provide a meaningful speedup.
  - Therefore, the optimal configuration for balancing speed and solution quality is a population size of 18 and a moderate number of generations (e.g., 6–8).

4.5.4 Conclusion

The proposed solution of reducing the population size to improve the speed of the **Genetic Algorithm** is not entirely satisfactory. While a population size of 15 or 18 provides a reasonable balance between speed and solution quality, it also leads to a loss of information, as smaller populations may not adequately explore the solution space. Furthermore, the gain in speed is not significant enough to justify the potential degradation in solution quality. Therefore, alternative approaches, such as parallelization or hybrid algorithms, should be explored to achieve a more meaningful improvement in computational efficiency without compromising the quality of the schedules.

5 Possible Improvements for the Genetic Algorithm

To enhance the performance of the **Genetic Algorithm (GA)**, several improvements can be implemented. These include parallelizing the fitness evaluation using a GPU and hybridizing the GA with other optimization techniques, such as **Local Search** or **Particle Swarm Optimization (PSO)**. Below, we describe each approach and provide pseudo-code for their implementation.

5.1 Parallelization Using a GPU

Parallelizing the fitness evaluation using a GPU can significantly reduce the computational time of the GA. GPUs are designed to handle thousands of threads simultaneously, making them ideal for evaluating the fitness of multiple individuals in parallel.

5.1.1 Pseudo-Code for GPU Parallelization

Algorithm 10 Parallel Fitness Evaluation Using a GPU

```
function EvaluateFitnessGPU(Population, Graph, Processors)
    Allocate memory on GPU for population and graph data
    Copy population and graph data to GPU
    Launch GPU kernel to evaluate fitness in parallel
    For each individual in the population:
        Calculate fitness using GPU threads
    Copy fitness results back to CPU
    return Fitness values
end function
```

5.1.2 Explanation

- The population and graph data are transferred to the GPU.
- A GPU kernel is launched to evaluate the fitness of each individual in parallel.
- Each thread on the GPU calculates the fitness of one individual.
- The results are copied back to the CPU for further processing.

5.2 Hybridization with Local Search

Hybridizing the GA with **Local Search** can improve the quality of solutions by refining the best individuals after each generation. Local Search explores the neighborhood of a solution to find better solutions.

5.2.1 Pseudo-Code for Hybrid GA with Local Search

Algorithm 11 Hybrid GA with Local Search

```
function HybridGAWithLocalSearch(Population, Generations)
    for each generation do
        Evaluate fitness of population
        Select parents and create offspring using crossover and mutation
        Apply Local Search to the best individuals:
            For each best individual:
                Explore neighborhood to find a better solution
                If a better solution is found, replace the individual
        Replace the population with the new offspring
    end for
    return Best solution found
end function
```

5.2.2 Explanation

- After each generation, the best individuals are refined using Local Search.

- Local Search explores the neighborhood of each individual to find better solutions.
- If a better solution is found, it replaces the original individual.

5.3 Hybridization with Particle Swarm Optimization (PSO)

Hybridizing the GA with **Particle Swarm Optimization (PSO)** can improve exploration and exploitation of the search space. PSO is a population-based optimization technique that uses the movement of particles to find optimal solutions.

5.3.1 Pseudo-Code for Hybrid GA with PSO

Algorithm 12 Hybrid GA with PSO

```
function HybridGAWithPSO(Population, Generations)
    Initialize PSO particles with GA population
    for each generation do
        Evaluate fitness of GA population and PSO particles
        Update PSO particles' velocities and positions
        Select parents from GA population and PSO particles
        Create offspring using crossover and mutation
        Replace the population with the new offspring
    end for
    return Best solution found
end function
```

5.3.2 Explanation

- The GA population is combined with PSO particles.
- PSO particles explore the search space using their velocities and positions.
- The best individuals from both GA and PSO are selected for reproduction.
- The population is updated with new offspring.

5.4 Conclusion

By parallelizing the fitness evaluation using a GPU and hybridizing the GA with Local Search or PSO, we can significantly improve the performance and solution quality of the Genetic Algorithm. These approaches leverage the strengths of different optimization techniques to achieve better results in less time.

6 Conclusion

In this article, we explored various scheduling algorithms for dependent tasks, focusing on their performance in terms of makespan and computational efficiency. Our experiments demonstrated that the **Blevel-based schedules** consistently outperform other approaches, providing the best balance between solution quality and execution speed. The proposed improvement using **Blevel with fuzzy interference** did not yield significant benefits, making its implementation unjustified in practice.

The **Genetic Algorithm (GA)** showed promise in achieving lower makespans, but its computational cost and variability in performance limit its applicability. Further investigation is required to optimize the GA, particularly through parallelization and hybridization with other optimization techniques, to make it competitive with Blevel-based schedules. Future work should also explore the integration of advanced heuristics and meta-heuristics to enhance the GA's efficiency and reliability.

In summary, while Blevel remains the most effective approach for task scheduling, the Genetic Algorithm offers potential for improvement and warrants further research to unlock its full capabilities.

References

- 1: ENS PSL MPI Exam, 2015
- 2: A novel task scheduling approach for dependent non-preemptive tasks using fuzzy logic

- 3: Standard Task Graphs Randomly Generated without Communication Costs
- 4: Multi-Heuristic List Scheduling Genetic Algorithms for Task Scheduling